
PILLAR

Hostinger VPS Deployment Guide

UAT / Pilot Environment
Configuration & Operations Manual

Version: 1.0.0

Date: July 2026

Environment: Hostinger VPS (UAT)

VPS: 72.60.104.187:2222

Target URL: <https://pillar.bayanaihan.net>

Deployment plan for UAT pilot testing of the PILLAR standalone application on Hostinger VPS before production deployment to Alibaba Cloud ECS.

Table of Contents

- 1. Deployment Overview**
Purpose, UAT vs. production comparison, architecture diagram
- 2. VPS Environment Assessment**
Server specs, port allocation, pre-existing infrastructure
- 3. ECS-to-VPS Adaptation Analysis**
Required changes, migration mapping
- 4. Deployment Procedure**
Step-by-step instructions, deployment script usage
- 5. Configuration Files Reference**
docker-compose.vps.yml, .env.vps, nginx details
- 6. UAT Testing Checklist**
Module testing, admin panel, LightRAG, performance
- 7. Operations & Maintenance**
Logs, restart, rebuild, backup, KB updates, SSL
- 8. Monitoring & Health Checks**
Endpoints, Docker health, resource monitoring
- 9. Troubleshooting**
Common issues and resolution steps
- 10. Post-Deployment Checklist**
Verification items after deployment
- A. VPS Configuration Quick Reference**
All settings at a glance
- B. File Inventory for Deployment**
Required files on VPS

PILLAR: Hostinger VPS Deployment Guide (UAT/Pilot)

Comprehensive Deployment Plan, Configuration & Operations Manual

Version: 1.0.0

Date: July 2026

Environment: Hostinger VPS (UAT/Pilot)

VPS Address: 72.60.104.187:2222

Target URL: <https://pillar.bayanaihan.net>

1. Deployment Overview

Purpose

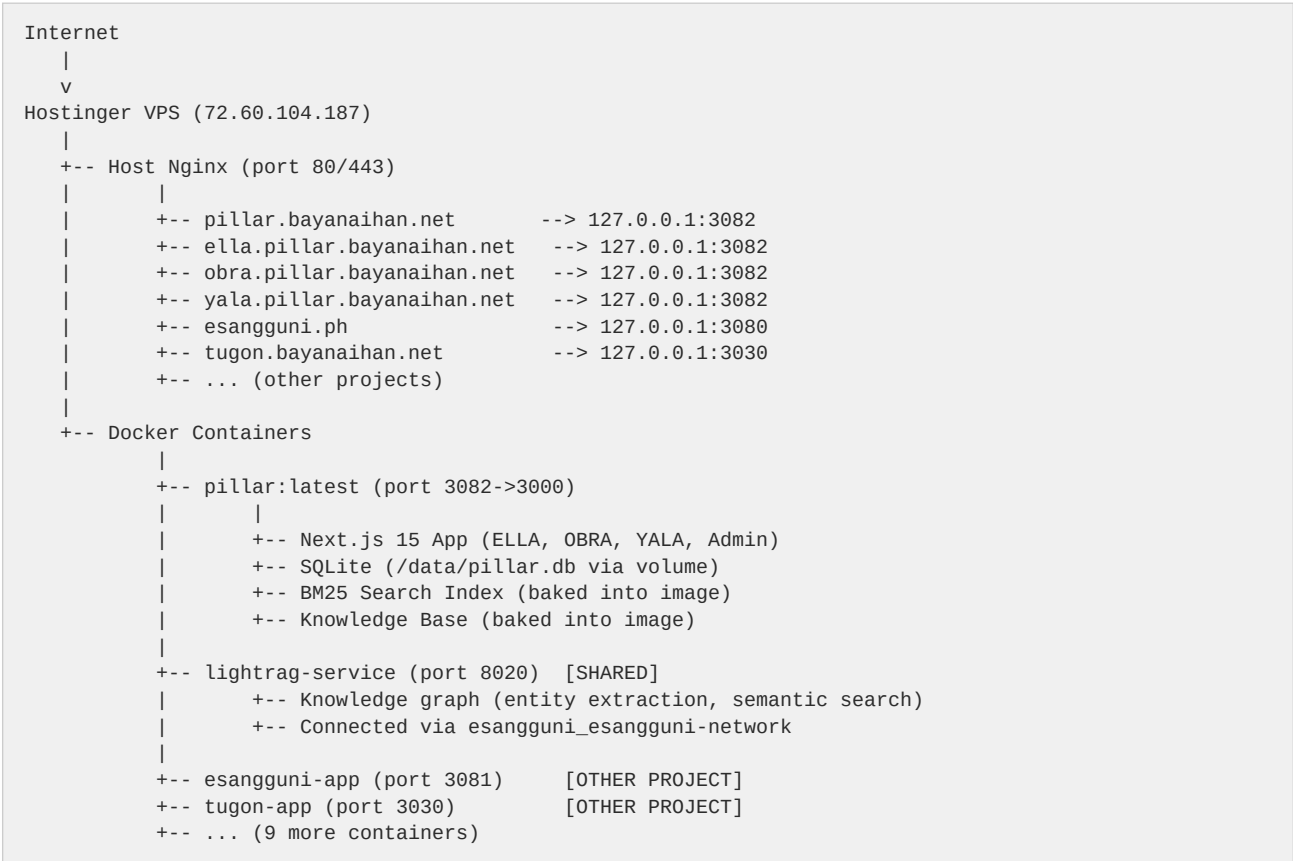
This document describes the deployment of the PILLAR standalone application to the Hostinger VPS for User Acceptance Testing (UAT) and pilot testing purposes. This environment serves as a staging ground to validate all features before the production deployment to Alibaba Cloud ECS.

UAT vs. Production (ECS)

Aspect	Hostinger VPS (UAT)	Alibaba Cloud ECS (Production)
Purpose	Pilot testing, feature validation, workshop dry-runs	Live deployment for multiple LGUs
Tenancy	Single PILLAR instance (Pitogo)	One container per LGU, multi-tenant
SSL/TLS	Host-level Let's Encrypt (existing)	Container-level Certbot (nginx profile)
LightRAG	Shared instance on <code>esangguni-network</code>	Dedicated per-province instance
Port	3082 (host nginx proxies from 443)	3000 (container nginx proxies)
Docker Compose	<code>docker-compose.vps.yml</code>	<code>docker-compose.yml</code>

Aspect	Hostinger VPS (UAT)	Alibaba Cloud ECS (Production)
Network	Joins existing <code>esangguni_esangguni-network</code>	Own <code>pillar-network</code> bridge
Resource Limits	2 GB RAM, 2 vCPUs (shared VPS)	Unrestricted (dedicated ECS)
eSANGGUNI Dependency	None (standalone)	None (standalone)
Admin Panel	Full admin interface available	Full admin interface available

Architecture on VPS



2. VPS Environment Assessment

Server Specifications

Resource	Value	Status
CPU	8 vCPUs	Sufficient
RAM	31 GB total, 3.0 GB used, 28 GB available	Plenty of headroom
Disk	387 GB total, 26 GB used, 361 GB available	Plenty of headroom
OS	Ubuntu (Linux)	Docker compatible
Docker	Installed, 12 containers running	Operational
Docker Compose	Available	Operational

Current Port Allocation

Port	Container	Project
3005	lightrag-admin	LightRAG Admin UI
3030	tugon-app	TUGON (Pulseboard)
3031	crmis	CRMIS
3033	subok-web	Subok
3080	esangguni-landing	eSANGGUNI Landing
3081	esangguni-app	eSANGGUNI App
3082	pillar (planned)	PILLAR
3100	leap100-chatbot	LEAP 100 Chatbot
3101	360growth-landing	360 Growth
8020	lightrag-service	LightRAG (shared)

Pre-Existing Infrastructure for PILLAR

The VPS already has several components in place from a previous PILLAR deployment:

Nginx Configuration (`/etc/nginx/sites-enabled/pillar`): Already configured with all four subdomain server blocks (pillar.bayanaihan.net, ella.pillar.bayanaihan.net, obra.pillar.bayanaihan.net, yala.pillar.bayanaihan.net). Each proxies to `127.0.0.1:3082` with SSE-optimized settings (300s read timeout, buffering off). SSL certificates are provisioned via Let's Encrypt.

SSL Certificates (`/etc/letsencrypt/live/pillar.bayanaihan.net/`): Valid Let's Encrypt certificates covering all four subdomains. Auto-renewal is managed by the host's Certbot timer.

LightRAG Service (port 8020): A shared LightRAG instance running on the `esangguni_esangguni-network` Docker network at container IP `172.16.10.6`. The new PILLAR container will join this network to access LightRAG by hostname.

Old Installation (`/opt/pillar-workshop/`): The previous monorepo-era PILLAR installation. This will be backed up and replaced with the new standalone codebase.

3. ECS-to-VPS Adaptation Analysis

Changes Required

The standalone PILLAR codebase was designed for ECS deployment. The following adaptations are necessary for the VPS environment:

1. Docker Compose File (`docker-compose.vps.yml`)

The ECS `docker-compose.yml` includes optional nginx and certbot services (gated behind the `production` Docker Compose profile) and binds to port 3000. For VPS deployment:

- **Port mapping changed:** `127.0.0.1:3082:3000` instead of `127.0.0.1:3000:3000`. The host nginx on the VPS handles SSL termination and proxies from port 443 to 3082.
- **Network bridging added:** The PILLAR container joins `esangguni_esangguni-network` (external) in addition to its own `pillar-network`. This enables hostname-based access to the shared LightRAG service.
- **Nginx/certbot services removed:** The VPS has a single host-level nginx managing all projects. Container-level nginx is unnecessary.
- **Resource limits added:** `2.0 cpus` and `2G memory` to prevent a single container from consuming too many shared resources.
- **Extended health check start period:** 60 seconds (up from 40s) to account for the build-from-source nature of VPS deployment.

2. Environment File (`.env.vps`)

- **DB_PATH changed:** `/data/pillar.db` (new standalone database, separate from the old `/data/pillar-workshop.db`).
- **WORKSHOP_SESSION_ID updated:** `2026-07-03-pillar-uat` to distinguish UAT interactions from production.
- **Public URLs set to full subdomains:** `https://ella.pillar.bayanaihan.net` etc., matching the nginx configuration.
- **ESANGGUNI_API_URL removed:** The standalone app has no eSANGGUNI dependency.
- **LIGHTRAG_SERVICE_URL set via compose:** Passed as `environment` in docker-compose rather than `.env` to ensure it uses the Docker network hostname.

3. Dockerfile (No Changes Required)

The existing multi-stage Dockerfile works as-is for VPS deployment. The build arguments are passed via the compose file's `args` section.

4. Nginx Configuration (No Changes Required)

The existing `/etc/nginx/sites-enabled/pillar` on the VPS already has the correct subdomain routing, SSL configuration, and SSE-optimized proxy settings. No modifications needed.

5. DNS Records (No Changes Required)

The DNS A records for `pillar.bayanaihan.net` and all subdomains (`ella.` , `obra.` , `yala.`) already point to `72.60.104.187` .

Migration from Old Installation

Old (<code>pillar-workshop</code>)	New (Standalone PILLAR)
Depended on <code>ESANGGUNI_API_URL</code>	Fully standalone, no external API
No admin interface	Full admin panel (KB, deployments, ECS)
No KB management	Dynamic document upload and indexing
Connected to <code>esangguni-network</code> for eSANGGUNI backend	Connected to <code>esangguni-network</code> for LightRAG only
DB at <code>/data/pillar-workshop.db</code>	DB at <code>/data/pillar.db</code> (fresh start)
Container name: <code>pillar-workshop</code>	Container name: <code>pillar</code>
Image: <code>pillar-workshop:latest</code>	Image: <code>pillar:latest</code>

4. Deployment Procedure

Prerequisites

Before deploying, ensure:

- 1. SSH access to the VPS at `72.60.104.187` on port `2222` as `root` .
- 2. The PILLAR codebase is available locally with all data files in `src/lib/data/documents/` (2,634 files) and `src/lib/data/search-index.json` (12 MB).
- 3. The `OPENROUTER_API_KEY` in `.env.vps` is valid and has sufficient credits.
- 4. The LightRAG service is running on the VPS (`docker ps | grep lightrag-service`).

Step-by-Step Deployment

Step 1: Backup Old Installation

```
ssh -p 2222 root@72.60.104.187
cd /opt
cp -r pillar-workshop pillar-workshop-backup-$(date +%Y%m%d-%H%M%S)
exit
```

Step 2: Stop Old Container

```
ssh -p 2222 root@72.60.104.187
cd /opt/pillar-workshop
docker compose down
exit
```

Step 3: Sync New Codebase

From the local PILLAR project directory:

```
ssh -p 2222 root@72.60.104.187 "mkdir -p /opt/pillar"
rsync -avz --delete \
  --exclude='node_modules' \
  --exclude='.next' \
  --exclude='.git' \
  --exclude='data/*.db*' \
  -e 'ssh -o StrictHostKeyChecking=no -p 2222' \
  ./ root@72.60.104.187:/opt/pillar/
```

Sync the environment file separately (it contains secrets):

```
rsync -avz -e 'ssh -p 2222' \
  .env.vps root@72.60.104.187:/opt/pillar/.env.vps
```

Step 4: Build and Start

```
ssh -p 2222 root@72.60.104.187
cd /opt/pillar

# Pull base image
docker pull node:20-alpine

# Build (takes 5-10 minutes on VPS)
docker compose -f docker-compose.vps.yml build --no-cache

# Start
docker compose -f docker-compose.vps.yml up -d

# Wait for health check (~60 seconds)
sleep 60

# Verify
docker ps --filter name=pillar --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
curl -s http://127.0.0.1:3082/api/health | python3 -m json.tool
```

Step 5: Verify All Subdomains

```
# Landing page
curl -sI https://pillar.bayanaihan.net | head -5

# ELLA
curl -sI https://ella.pillar.bayanaihan.net | head -5

# OBRA
curl -sI https://obra.pillar.bayanaihan.net | head -5

# YALA
curl -sI https://yala.pillar.bayanaihan.net | head -5
```

All should return `HTTP/2 200` or `HTTP/2 301` (subdomain redirects).

Step 6: Verify LightRAG Integration

```
# From VPS host
curl -s http://127.0.0.1:8020/health

# From PILLAR container (via Docker exec)
docker exec pillar wget -qO- http://lightrag-service:8020/health 2>/dev/null
```

If LightRAG is healthy, PILLAR will use hybrid search (BM25 + graph). If not, it falls back to BM25-only.

Using the Deployment Script

A shell script (`deploy-vps.sh`) automates the full procedure:

```
# Full deployment (backup + sync + build + start)
chmod +x deploy-vps.sh
./deploy-vps.sh deploy

# Individual commands
./deploy-vps.sh backup      # Backup old installation
./deploy-vps.sh sync       # Sync code only
./deploy-vps.sh stop-old   # Stop old container
./deploy-vps.sh build      # Build and start new
./deploy-vps.sh restart    # Restart container
./deploy-vps.sh rollback   # Roll back to old version
./deploy-vps.sh status     # Check status
./deploy-vps.sh logs 100   # View last 100 log lines
```

5. Configuration Files Reference

`docker-compose.vps.yml`

Key configuration decisions:

Port 3082: Chosen to avoid conflict with the existing `pillar-workshop` port (also 3082) — the old container must be stopped before the new one can bind to this port.

Dual Network Membership: The `pillar` service connects to both `pillar-network` (its own bridge) and `esangguni_esangguni-network` (external). This is required because LightRAG lives on the eSANGGUNI network. Docker allows containers to be on multiple networks simultaneously.

LIGHTRAG_SERVICE_URL via `environment`: Rather than placing this in `.env.vps`, it's set directly in the compose file's `environment` section. This ensures the value uses the Docker network hostname (`lightrag-service`) rather than a host IP, making it resilient to LightRAG container restarts.

Resource Limits: 2 CPUs and 2 GB RAM are sufficient for a single-LGU PILLAR instance. The Next.js standalone server typically uses 100-200 MB RAM at idle, with spikes during LLM streaming and document ingestion.

`.env.vps`

OpenRouter API Key: Reuses the existing key from the old `pillar-workshop` installation. This key has been validated and has sufficient credits.

Workshop Session ID: Set to `2026-07-03-pillar-uat` to clearly distinguish UAT interactions from any production workshop data. Change this before each new UAT testing session.

Database Path: `/data/pillar.db` — a fresh database for the standalone app. The old `pillar-workshop.db` is preserved in the `pillar-workshop_pillar-data` Docker volume and in the backup directory.

Host Nginx (`/etc/nginx/sites-enabled/pillar`)

The existing nginx configuration is correct and does not need changes. Key settings already in place:

- **Four server blocks:** One per subdomain, all proxying to `http://127.0.0.1:3082`.
- **SSE optimization:** `proxy_read_timeout 300s` and `proxy_buffering off` for AI chat streaming.
- **WebSocket upgrade:** `$http_upgrade` and `Connection 'upgrade'` headers.
- **SSL:** Let's Encrypt certificates with HSTS headers.
- **Subdomain redirects:** `ella.pillar.bayanaihan.net/` redirects to `/ella`, etc.

6. UAT Testing Checklist

Core Module Testing

ELLA (Legal Research)

1. Navigate to `https://ella.pillar.bayanaihan.net/ella`.
2. Submit a legal research query (e.g., "What are the powers of the Sangguniang Bayan under Section 447?").
3. Verify streaming response with real-time text display.

4. Check that citations are present and verified (green checkmark).
5. Verify citation links open the full document text.
6. Test follow-up questions within the same conversation.

OBRA (Ordinance Drafting)

1. Navigate to <https://obra.pillar.bayanaihan.net/obra> .
2. Select a template type (e.g., "Regulatory").
3. Fill in draft details (title, subject matter, key provisions).
4. Generate a draft and verify the output structure (Title, Whereas, Body, Penal, Effectivity).
5. Check citation warnings for any unverified references.
6. Test the compliance review feature with the generated draft.
7. Verify the review output includes score, risks, structural issues, and missing sections.

YALA (Citizen Information)

1. Navigate to <https://yala.pillar.bayanaihan.net/yala> .
2. Ask about municipal services (e.g., "How do I get a business permit?").
3. Verify bilingual responses (English and Filipino).
4. Test out-of-scope questions (e.g., "What's the weather?") and verify polite refusal.
5. Check FAQ integration.

Admin Panel Testing

1. Navigate to <https://pillar.bayanaihan.net/admin/login> .
2. Login with default credentials (`admin` / `pillar-admin-2026`).
3. Verify redirect to admin dashboard.
4. Test Knowledge Base management:
 - View document list, filters, and pagination.
 - Upload a test document.
 - Trigger re-index.
 - Delete the test document.
5. Test analytics dashboard with workshop data.
6. Verify deployment and ECS management pages (visible to super_admin only).

LightRAG Integration Testing

1. Check health endpoint: `GET /api/health` should show `"lightrag": "healthy"` .
2. Submit an ELLA query and verify the response includes graph-based context (look for `[HYBRID]` in the context header, visible in browser dev tools).
3. If LightRAG is unavailable, verify graceful degradation (response should still work with BM25-only, marked as `[DEGRADED MODE]`).

Performance Testing

1. Monitor response times for ELLA queries (target: under 5 seconds for first token).
2. Check OBRA draft generation time (target: under 30 seconds).
3. Monitor container resource usage via `docker stats pillar`.
4. Verify health check endpoint responds within 1 second.

7. Operations & Maintenance

Routine Operations

Viewing Logs:

```
ssh -p 2222 root@72.60.104.187
docker logs pillar --tail 100 -f
```

Restarting the Application:

```
cd /opt/pillar
docker compose -f docker-compose.vps.yml restart
```

Stopping the Application:

```
cd /opt/pillar
docker compose -f docker-compose.vps.yml down
```

Rebuilding After Code Changes:

```
# Sync updated code
rsync -avz --delete \
  --exclude='node_modules' --exclude='.next' --exclude='.git' --exclude='data/*.db*' \
  -e 'ssh -p 2222' \
  ./ root@72.60.104.187:/opt/pillar/

# Rebuild and restart
ssh -p 2222 root@72.60.104.187 "cd /opt/pillar && docker compose -f docker-compose.vps.yml up -d --build"
```

Data Management

Backing Up the SQLite Database:

```
ssh -p 2222 root@72.60.104.187
docker exec pillar cp /data/pillar.db /data/pillar-backup-$(date +%Y%m%d).db
docker cp pillar:/data/pillar-backup-$(date +%Y%m%d).db /opt/backups/
```

Resetting the Database (Fresh Start):

```
ssh -p 2222 root@72.60.104.187
docker compose -f docker-compose.vps.yml down
docker volume rm pillar_pillar-data
docker compose -f docker-compose.vps.yml up -d --build
```

Note: This deletes all workshop data, admin users, and KB documents. The default admin account will be re-seeded on first startup.

Changing the Workshop Session:

Edit `.env.vps` and update `WORKSHOP_SESSION_ID` :

```
ssh -p 2222 root@72.60.104.187
cd /opt/pillar
nano .env.vps
# Change WORKSHOP_SESSION_ID=2026-MM-DD-new-session-name
docker compose -f docker-compose.vps.yml restart
```

A restart is sufficient (no rebuild needed) since the session ID is read at runtime.

Updating the Knowledge Base

The BM25 search index and document files are baked into the Docker image at build time. To update the knowledge base:

1. **Locally:** Run `npm run ingest` to regenerate the search index and document files.
2. **Sync:** Push updated files to the VPS via rsync.
3. **Rebuild:** `docker compose -f docker-compose.vps.yml up -d --build` .

For dynamic KB additions at runtime, use the admin upload interface. Uploaded documents are stored in SQLite and ingested into LightRAG via re-index, but they will NOT appear in BM25 search results until the image is rebuilt with the updated index.

SSL Certificate Renewal

SSL certificates are managed by the host's Certbot timer, not by the PILLAR container. Verify renewal:

```
ssh -p 2222 root@72.60.104.187
certbot certificates | grep pillar
```

If renewal fails, manually renew:

```
certbot renew --cert-name pillar.bayanaihan.net
systemctl reload nginx
```

Rollback Procedure

If the new standalone PILLAR has issues, roll back to the old installation:

```
ssh -p 2222 root@72.60.104.187

# Stop new container
cd /opt/pillar
docker compose -f docker-compose.vps.yml down

# Start old container
cd /opt/pillar-workshop
docker compose up -d
```

The old installation uses the same port (3082) and nginx configuration, so it will immediately become accessible again.

8. Monitoring & Health Checks

Health Endpoints

Endpoint	Purpose	Expected Response
GET /api/health	PILLAR service health	<pre>{"status": "ok", "modules": ["ella", "obra", "yala"], "lightrag": "healthy"}</pre>
GET http://127.0.0.1:8020/health	LightRAG service health	<pre>{"status": "healthy", "rag_loaded": true}</pre>

Docker Health Check

The container has a built-in Docker health check that runs every 30 seconds:

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:3000/api/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 60s
```

Monitor health status:

```
docker inspect pillar --format '{{.State.Health.Status}}'
```

Resource Monitoring

```
# Real-time resource usage
docker stats pillar --no-stream

# Disk usage of Docker volumes
docker system df -v | grep pillar
```

Log Analysis

```
# Error logs only
docker logs pillar 2>&1 | grep -i error

# API request logs
docker logs pillar 2>&1 | grep -E "(GET|POST)" | tail -20

# LLM response times
docker logs pillar 2>&1 | grep -i "response_time"
```

9. Troubleshooting

Container Won't Start

```
# Check logs
docker logs pillar --tail 50

# Common causes:
# 1. Port 3082 already in use (old container still running)
docker ps -a --filter publish=3082
docker stop pillar-workshop # stop the old one

# 2. Build failure (native compilation of better-sqlite3)
docker compose -f docker-compose.vps.yml build --no-cache 2>&1 | tail -30

# 3. Missing .env.vps
ls -la /opt/pillar/.env.vps
```

LightRAG Not Reachable

```
# Check if PILLAR is on the right network
docker inspect pillar --format '{{json .NetworkSettings.Networks}}' | python3 -m json.tool

# Check LightRAG health from inside the PILLAR container
docker exec pillar wget -qO- http://lightrag-service:8020/health

# If network issue, reconnect
docker network connect esangguni_esangguni-network pillar
```

SSE Streaming Not Working

```
# Test SSE from VPS host
curl -N -H "Content-Type: application/json" \
  -d '{"module":"ella","message":"test","history":[],"sessionId":"test"}' \
  http://127.0.0.1:3082/api/chat

# If streaming works locally but not externally, check nginx config
grep -A5 'proxy_buffering' /etc/nginx/sites-enabled/pillar
# Should show: proxy_buffering off;
```

Admin Login Not Working

```
# Check if admin user exists in database
docker exec pillar sqlite3 /data/pillar.db "SELECT username, role, is_active FROM admin_users;"

# If empty, the seed didn't run. Check logs:
docker logs pillar 2>&1 | grep -i seed

# Manual admin creation (via SQLite inside container):
docker exec -it pillar sqlite3 /data/pillar.db
# Then run the seed SQL manually
```

High Memory Usage

```
# Check current usage
docker stats pillar --no-stream

# If exceeding 2GB limit, check for memory leaks
docker exec pillar cat /proc/1/status | grep VmRSS

# Restart if needed
docker compose -f docker-compose.vps.yml restart
```

10. Post-Deployment Checklist

After successful deployment, verify each item:

- [☐] <https://pillar.bayanaihan.net> loads the landing page
- [☐] <https://ella.pillar.bayanaihan.net> loads ELLA and chat works
- [☐] <https://obra.pillar.bayanaihan.net> loads OBRA and draft generation works
- [☐] <https://yala.pillar.bayanaihan.net> loads YALA and chat works
- [☐] <https://pillar.bayanaihan.net/admin> login works with default credentials
- [☐] Health endpoint returns `"status": "ok"` and `"lightrag": "healthy"`
- [☐] ELLA citations are verified (green checkmarks)
- [☐] OBRA compliance review returns structured JSON
- [☐] YALA refuses out-of-scope questions politely
- [☐] Default admin password has been changed
- [☐] Docker health check shows "healthy"
- [☐] Container resource usage is within limits
- [☐] Old pillar-workshop backup is confirmed

Appendix A: VPS Configuration Quick Reference

Setting	Value
VPS IP	72.60.104.187
SSH Port	2222
SSH User	root
Project Directory	/opt/pillar
Docker Compose File	docker-compose.vps.yml
Container Name	pillar
Image Name	pillar:latest
Host Port	3082
Container Port	3000
Database Path	/data/pillar.db
Docker Volume	pillar_pillar-data
Network (own)	pillar_pillar-network
Network (shared)	esangguni_esangguni-network
LightRAG URL	http://lightrag-service:8020
Nginx Config	/etc/nginx/sites-enabled/pillar
SSL Cert	/etc/letsencrypt/live/pillar.bayanaihan.net
CPU Limit	2.0 vCPUs
Memory Limit	2 GB

Appendix B: File Inventory for Deployment

Files that must be present on the VPS at `/opt/pillar/` :

```
/opt/pillar/
Dockerfile                # Multi-stage build (unchanged from ECS version)
docker-compose.vps.yml    # VPS-adapted compose (NEW)
.env.vps                  # VPS environment variables (NEW)
.dockerignore             # Docker build exclusions
next.config.ts            # Next.js config (output: "standalone")
package.json              # Dependencies
package-lock.json         # Lock file
postcss.config.mjs        # PostCSS config
tailwind.config.ts        # Tailwind config
tsconfig.json             # TypeScript config
public/                  # Static assets
src/                      # Application source
  app/                   # Next.js App Router pages and API routes
  lib/                   # Backend libraries
  data/                  # Knowledge base data
    documents/           # 2,634 document JSON files
    search-index.json    # BM25 index (12 MB)
```

```
yala-knowledge-base.json
faq-data.json
...
ai/                # AI backend (RAG, LLM, prompts)
db.ts              # SQLite setup
auth.ts            # Authentication
...
types/             # TypeScript type definitions
```

This document was generated for the PILLAR UAT deployment to Hostinger VPS. For the production ECS deployment, refer to PILLAR-Documentation.md.